



Universidad Nacional de Loja

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO
RENOVABLES

COMPUTACIÓN

Kleyner Buri

Asignatura: Estructuras de Datos

Investigación de Algoritmos que Evitan el Reprocesamiento

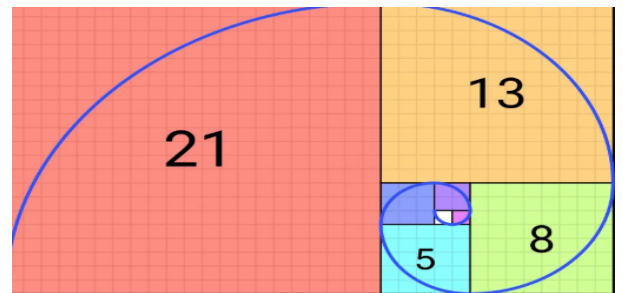
Fase 1: Análisis Teórico y Diseño Lógico

1.1 Identificación del problema: Serie de Fibonacci recursivo

Para esta práctica se utilizó la serie de Fibonacci como problema de estudio. Esta serie matemática se caracteriza porque cada número se obtiene sumando los dos números anteriores, comenzando con 0 y 1. Se eligió este problema porque su implementación recursiva realiza muchos cálculos repetidos, lo que provoca un aumento considerable en el tiempo de ejecución cuando el valor de entrada crece.

Debido a esta característica, la serie de Fibonacci permite observar claramente la diferencia entre un algoritmo tradicional y otro optimizado mediante memorización. Para ello, se almacenan los resultados ya calculados en una estructura hash, evitando repetir operaciones innecesarias. De esta forma, se puede analizar cómo el uso de memoria adicional ayuda a reducir la carga de procesamiento de la CPU y mejorar el rendimiento del programa.

$$F(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ F(n-1) + F(n-2) & \text{si } n > 1 \end{cases}$$



1.2 Modelado de Almacenamiento: Para evitar el reprocesamiento de cálculos repetidos, se diseñó una estructura de almacenamiento temporal basada en un `HashMap<Integer, Long>`. Esta estructura funciona como una memoria caché de corto plazo, donde la clave `Integer` representa el valor de entrada de la función Fibonacci y el valor `Long` almacena el resultado calculado para esa entrada.

Antes de realizar un nuevo cálculo recursivo el algoritmo verifica si el resultado ya existe dentro del `HashMap`. Si el valor está almacenado, se recupera directamente desde la memoria caché; caso contrario, el algoritmo realiza el cálculo, guarda el resultado en la estructura hash y posteriormente lo devuelve al programa.

De esta manera, cada valor de Fibonacci se calcula una única vez, reduciendo significativamente la cantidad de operaciones ejecutadas y mejorando la eficiencia del algoritmo.

- **Complejidad temporal:** La complejidad temporal es de $O(2^n)$ donde $2 = 1.618$ que corresponde a la proporción áurea. Esto implica un crecimiento exponencial del número de operaciones a medida que aumenta el valor de n .
- **Complejidad espacial:** La complejidad espacial de este algoritmo es de $O(n)$ debido a la profundidad máxima alcanzada por la pila de llamadas *call stack* durante la ejecución recursiva.

2.0 Fase 2: Implementación Experimental (Java o Python)

2.1 Implementación de Clase Utilidades: La clase `Utilidades` contiene la implementación de los algoritmos utilizados en el experimento. Incluye una versión recursiva tradicional de la serie de Fibonacci y una versión optimizada mediante memoización o optimización informática utilizando un `HashMap` como memoria caché para evitar cálculos repetidos.

```

package edu.unl.cc;

import java.util.HashMap;

/**
 * @author kleynerbuuzz
 */
public class Utilidades {
    private HashMap<Integer, Long> cache = new HashMap<>();

    //Fibonacci recursivo
    public long fibonacciRekursivo(int n) {
        if (n <= 1) {
            return n;
        }
        return fibonacciRekursivo(n - 1) + fibonacciRekursivo(n - 2);
    }

    //Fibonacci con memoria
    public long fibonacciMemo(int n) {
        if (n <= 1) {
            return n;
        }
        if (cache.containsKey(n)) {
            return cache.get(n);
        }
        long resultado = fibonacciMemo(n - 1) + fibonacciMemo(n - 2);
        cache.put(n, resultado);
        return resultado;
    }

    public void limpiarCache() { cache.clear(); }
}

```

- **Implementación de clase Ejecutor:** La clase Ejecutor contiene el método principal main encargado de ejecutar las pruebas experimentales. Su función es llamar a ambos algoritmos, medir sus tiempos de ejecución mediante System.nanoTime() y mostrar los resultados obtenidos para su posterior análisis.

```

1 package edu.unl.cc;
2
3 /**
4  * @author kleyner buzz
5  */
6
7 public class Ejecutor {
8     public static void main(String[] args) {
9         Utilidades u = new Utilidades();
10        int[] pruebas = {10, 20, 30, 35, 40};
11
12        System.out.println("N\tRecursivo(ns)\tMemoizacion(ns)");
13
14        for (int n : pruebas) {
15            long inicio = System.nanoTime();
16            long resultadoRecursivo = u.fibonacciRecursivo(n);
17            long fin = System.nanoTime();
18            long tiempoRecursivo = fin - inicio;
19
20            u.limpiarCache();
21            inicio = System.nanoTime();
22
23            long resultadoMemo =
24                u.fibonacciMemo(n);
25            fin = System.nanoTime();
26
27            long tiempoMemo = fin - inicio;
28            System.out.println(
29                n + "\t" + tiempoRecursivo + "\t\t" + tiempoMemo
30            );
31        }
32    }
33 }
34
35

```

- Ejecución:** Ambos algoritmos resuelven el mismo problema y producen el mismo resultado. La diferencia es que la versión recursiva vuelve a calcular varias veces los mismos valores, mientras que la versión con memoización guarda los resultados obtenidos en un [HashMap](#) para reutilizarlos cuando sean necesarios. Esto evita realizar cálculos repetidos y permite reducir significativamente el tiempo de ejecución del programa.

```

↑ /usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/snap/intellij-idea-u
Build ↓ Recursivo(ns) Memoizacion(ns)
10 4997 74645
20 177286 26735
30 3611194 39985
35 36639830 37709
40 413802459 57357

Process finished with exit code 0

```

3.0 Fase 3: Protocolo de Medición y Perfilado Físico

3.1 Ejecute ambos enfoques: Con tamaños de entrada incrementales (N = 10, N = 20, N = 30, etcétera).

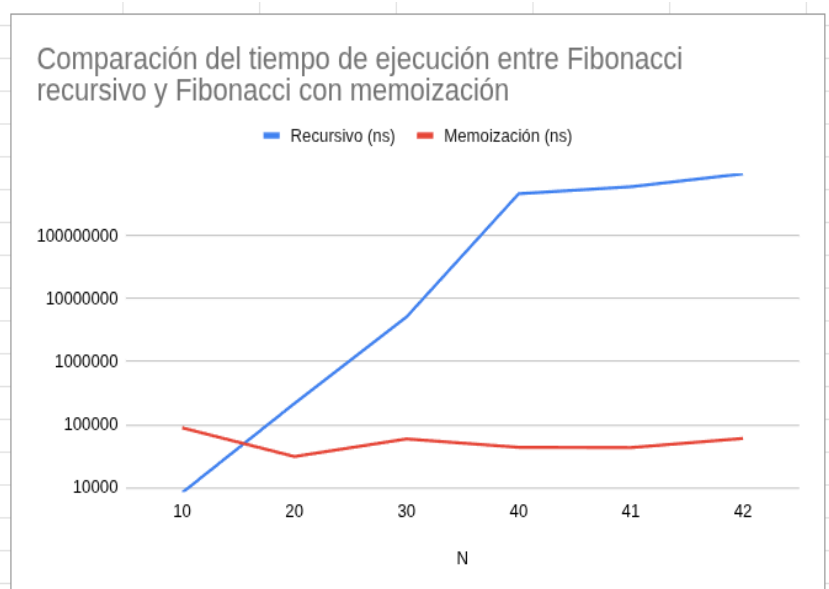
3.2 Registre la latencia acumulada: Utilizando mediciones de precisión extrema (System.nanoTime() en Java o time.perf_counter_ns() en Python).

```
↑ /usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/snap/intellij-idea-U
↓
≡
≡
≡
≡
≡
≡
Process finished with exit code 0
```

N	Recursivo(ns)	Memoizacion(ns)
10	8224	88618
20	215915	30946
30	5148190	58978
40	469959395	43373
41	607223913	42907
42	975102069	60119

3.3 Grafique de forma superpuesta las curvas de rendimiento: Tamaño de Entrada (N) versus Tiempo de Ejecución (ns) para identificar el punto exacto de colapso de la CPU en el enfoque ineficiente.

N	Recursivo (ns)	Memoización (ns)
10	8224	88618
20	215915	30946
30	5148190	58978
40	469959395	43373
41	607223913	42907
42	975102069	60119



4.0 Preguntas de Control:

- ¿Cuál es el compromiso (trade-off) espacial de los algoritmos de prevención de reprocesamiento? Explique por qué sacrificamos espacio en la memoria RAM para salvar ciclos en la CPU.

El compromiso consiste en usar más memoria RAM para almacenar resultados que ya fueron calculados y así evitar repetir el mismo trabajo, en lugar de volver a ejecutar un cálculo, el algoritmo simplemente consulta el valor guardado. Esto hace que el programa sea mucho más eficiente. Aunque se utiliza más memoria, se reduce considerablemente el tiempo de ejecución. Con esto acceder a un dato almacenado es mucho más rápido que recalcularlo varias veces, por lo que la CPU realiza menos operaciones y el algoritmo pasa de un tiempo exponencial a uno lineal.

- Si una estructura de tipo Tabla Hash sufre de una alta tasa de colisiones, ¿cómo se ve afectada la eficiencia del algoritmo de memoización y qué impacto colateral tiene en la latencia del sistema?

Si una Tabla Hash tiene muchas colisiones, encontrar un dato deja de ser una operación rápida. En vez de acceder directamente al valor, el programa debe recorrer más elementos para localizar la clave correcta, lo que aumenta el tiempo de búsqueda. Esto afecta directamente a la memoización, ya que cada consulta tarda más de lo esperado. Como el algoritmo realiza muchas búsquedas en la tabla, esos pequeños retrasos se acumulan y provocan un aumento en la latencia del sistema, haciendo que el rendimiento general disminuya.

- Desde la perspectiva de la arquitectura de hardware, ¿cómo influye el almacenamiento en la memoria caché del procesador (L1, L2, L3) cuando el algoritmo recupera de forma interactiva valores guardados en lugar de recalcularlos en la Unidad Aritmético Lógica (ALU)?

Cuando el algoritmo consulta varias veces los mismos datos, es muy probable que esos valores se almacenen en la memoria caché del procesador (L1, L2 o L3). Si el dato se encuentra allí, la CPU puede recuperarlo en muy pocos ciclos de reloj, lo que acelera significativamente la ejecución, pero si el dato no está en la caché y debe buscarse en la RAM, el acceso será más lento, pero a pesar de esto recuperar un resultado ya almacenado sigue siendo mucho más eficiente que volver a realizar todos los cálculos en la ALU, especialmente en algoritmos con muchas operaciones repetitivas como Fibonacci.

3. REFERENCIAS

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] R. Sedgewick y K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[3] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA, USA: Pearson, 2012.

[4] GeeksforGeeks, "Sorting Algorithms," 2026. [En línea]. Disponible: <https://www.geeksforgeeks.org/sorting-algorithms/> [Accedido: 28-may-2026].